



Testing, Debugging y Documentación con IA (70 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

➤ Generación de documentación técnica y comentarios de código.



- I. ¿Por qué documentar con IA?
- II. Tipos de documentación que puede generar la IA
- III. Cómo pedir documentación a la IA
- IV. Ejemplo real: Docstring generado por IA
- V. Documentación de APIs con IA
- VI. IA explicando código heredado
- VII. Buenas prácticas al documentar con IA y errores comunes



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

➤ Generación de documentación técnica y comentarios de código.



Objetivos de este módulo

Al finalizar este módulo, serás capaz de:

1. ****Identificar**** los diferentes tipos de documentación técnica (interna, externa, automática).
2. ****Utilizar**** la IA de forma efectiva para generar documentación de calidad.
3. ****Construir prompts**** detallados y contextualizados para obtener los mejores resultados.
4. ****Evaluar**** la documentación generada por IA, identificando errores y áreas de mejora.
5. ****Aplicar**** buenas prácticas para integrar la documentación generada por IA en un flujo de trabajo profesional.
6. ****Reutilizar**** la IA para entender y documentar código heredado complejo

➤ Generación de documentación técnica y comentarios de código.



🤔 ¿Alguna vez has tenido que...?

- * ...volver a tu código después de 6 meses y no entender qué hacía una función?
- * ...heredar un proyecto sin una sola línea de comentario?
- * ...pasar horas escribiendo documentación repetitiva y aburrida?

Imagina que la IA es tu ****"traductor personal"**.**

Puede tomar tu código (o el de otros) y convertirlo en una historia clara y bien estructurada.

Vamos a ser el jefe de ese traductor, dándole las instrucciones precisas para que genere exactamente la documentación que necesitas, cuando la necesitas.



Idea clave

La integración de herramientas de IA en el proceso de documentación técnica y comentado de código o programas no solo acelera la creación de contenido, sino que transforma la manera en que los trabajadores o programadores de las empresas (y desarrolladores en general) entienden, estructuran y comunican su trabajo.

Esta capacidad potencia tanto el aprendizaje individual como la colaboración en equipo, todo ello conlleva a la reducción de la carga de trabajo asociada a los proyectos.

3.1 — ¿Por qué documentar con IA?

La documentación es una de las tareas más olvidadas por los profesionales y, al mismo tiempo, una de las más críticas en cualquier proyecto profesional.

La IA convierte este proceso en algo más rápido, claro y accesible, ayudando a que la documentación deje de ser un obstáculo y se convierta en una herramienta de aprendizaje y mantenimiento. Los motivos son los siguientes:

◆ Ahorra tiempo

La IA puede generar documentación en segundos:

- descripciones de funciones y módulos
- explicaciones de parámetros y retornos
- ejemplos de uso
- advertencias y notas técnicas
- comentarios en el propio código

Esto libera tiempo para que los profesionales se centren en comprender la lógica y mejorar el diseño del software.

◆ Mejora la claridad del código

La IA ayuda a transformar código complejo en explicaciones comprensibles:

- resume lógica complicada
- explica algoritmos paso a paso
- aclara dependencias
- describe flujos internos
- señala decisiones de diseño

Esto facilita que cualquier persona —incluido el propio programador semanas después— entienda qué hace el código y por qué.

◆ Facilita el mantenimiento

Una documentación clara reduce:

- errores al modificar código
- malentendidos entre miembros del equipo
- tiempo perdido buscando cómo funciona algo
- riesgo de romper funcionalidades existentes

La IA actúa como un asistente que mantiene la coherencia documental del proyecto.

◆ Ayuda a entender código heredado

La IA puede analizar código:

- antiguo
- mal documentado
- escrito por otra persona
- generado automáticamente
- con lógica poco intuitiva

Y producir:

- resúmenes
- explicaciones pedagógicas
- diagramas conceptuales en texto
- descripciones de flujos

Esto es especialmente útil en entornos educativos donde los alumnos trabajan con proyectos previos.

Generación de documentación técnica y comentarios de código.



◆ Genera documentación consistente

La IA mantiene un estilo uniforme:

- estructura repetible
- formato coherente
- terminología estándar
- claridad en los nombres
- organización profesional

Esto evita que la documentación parezca escrita por varias personas con estilos distintos.



◆ Enseña buenas prácticas de redacción técnica

La IA puede mostrar ejemplos profesionales de:

- cómo escribir descripciones claras
- cómo documentar parámetros y retornos
- cómo redactar comentarios útiles (no redundantes)
- cómo estructurar documentación de APIs
- cómo incluir advertencias de seguridad o rendimiento

Así aprendemos observando documentación bien hecha.

Generación de documentación técnica y comentarios de código.



- ◆ **Acelera el onboarding de nuevos miembros**

Un nuevo desarrollador en el equipo puede pasar días o semanas tratando de entender la arquitectura y el flujo del proyecto.

La IA puede generar una descripción general del sistema, los módulos principales y sus interacciones en cuestión de minutos, reduciendo drásticamente la curva de aprendizaje.



Problema real sin documentación

```
def procesar(x, y):  
    return x * y / 2
```

- ¿Qué hace esta función?
- ¿Qué significa x?
- ¿Qué devuelve?

Ejemplo de cada tipo

- Comentarios en línea
- Docstrings
- README
- Documentación API

👉 Muy visual (lista + mini ejemplos)

```
def procesar(x, y):  
    """  
    Calcula el área de un triángulo.  
  
    Parámetros:  
    x (float): base  
    y (float): altura  
  
    Retorna:  
    float: área del triángulo  
    """  
    return x * y / 2
```

3.2 — Tipos de documentación que puede generar la IA

✓ Documentación interna

Documentación que vive *dentro del código* y explica su funcionamiento desde dentro. Es esencial para la mantenibilidad y para que otros desarrolladores entiendan la lógica.

◆ Comentarios en línea

La IA puede añadir comentarios breves y útiles que expliquen:

- ☐ qué hace una línea o bloque
- ☐ por qué se toma una decisión
- ☐ qué casos especiales se contemplan
- ☐ qué riesgos se evitan

Ejemplo:

```
// Filtra usuarios activos para evitar procesar cuentas deshabilitadas
```



3.2 — Tipos de documentación que puede generar la IA



Documentación orientada a roles



La IA puede adaptar el nivel y el enfoque de la documentación según quién la vaya a leer:

- ****Para un desarrollador backend:**** Explicar la lógica de negocio, las dependencias y la seguridad de un endpoint.
- ****Para un desarrollador frontend:**** Enfocarse en la estructura de la respuesta JSON, los códigos de error y cómo consumir la API.
- ****Para un QA (tester):**** Detallar los casos limite, los valores de entrada válidos e inválidos y los posibles errores para crear casos de prueba.
- ****Para un DevOps:**** Explicar las variables de entorno necesarias, los puertos, las dependencias del sistema y los scripts de despliegue.

3.2 — Tipos de documentación que puede generar la IA

◆ Docstrings

Documentación formal dentro de funciones, clases o métodos:

- ☐ descripción clara
- ☐ parámetros
- ☐ tipos
- ☐ valores de retorno
- ☐ excepciones
- ☐ ejemplos de uso

Ejemplo (Python):

```
def get_user(id):  
    """  
    Devuelve un usuario por ID.  
    Args:  
        id (int): Identificador del usuario.  
    Returns:  
        User | None: Usuario encontrado o None si no existe.  
    """
```

Generación de documentación técnica y comentarios de código.



3.2 — Tipos de documentación que puede generar la IA

◆ Explicación de funciones y clases

La IA puede generar descripciones completas:

- ☐ propósito
- ☐ comportamiento
- ☐ dependencias
- ☐ efectos secundarios
- ☐ limitaciones

Esto ayuda a los programadores a entender código complejo o heredado.

```
def dividir(a, b):  
    return a / b
```



Generación de documentación técnica y comentarios de código.

```
def dividir(a, b):  
    return a / b
```



```
def dividir(a, b):  
    """  
    Divide dos números.  
    Parámetros:  
    -----  
    a : float o int  
        Numerador (número a dividir).  
    b : float o int  
        Denominador (número por el que se divide).  
    Retorna:  
    -----  
    float  
        Resultado de la división a ÷ b.  
    Lanza:  
    -----  
    ZeroDivisionError  
        Si b es igual a 0 (no se puede dividir por cero).  
    TypeError  
        Si a o b no son números (int o float).  
    Ejemplos:  
    -----  
    >>> dividir(10, 2)  
    5.0  
    >>> dividir(7, 3)  
    2.3333333333333335  
    >>> dividir(5, 0)  
    ZeroDivisionError: No se puede dividir por cero  
    Nota:  
    -----  
    En Python, dividir por cero produce ZeroDivisionError.  
    """  
    if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):  
        raise TypeError("Ambos parámetros deben ser números")  
    if b == 0:  
        raise ZeroDivisionError("No se puede dividir por cero")  
    return a / b
```



3.2 — Tipos de documentación que puede generar la IA

✓ Documentación externa

Documentación que vive *fuera del código* y sirve para comunicar el proyecto a usuarios, compañeros o evaluadores.

◆ README

La IA puede generar un README profesional con:

- ☐ descripción del proyecto
- ☐ instalación
- ☐ uso
- ☐ ejemplos
- ☐ dependencias
- ☐ estructura del repositorio
- ☐ notas de seguridad

Perfecto para enseñar buenas prácticas de repositorios.

Generación de documentación técnica y comentarios de código.



■ 3.2 — Tipos de documentación que puede generar la IA

◆ Manuales técnicos

Documentación más extensa:

- ☐ arquitectura
- ☐ decisiones de diseño
- ☐ diagramas conceptuales (texto)
- ☐ flujos de datos
- ☐ requisitos técnicos
- ☐ limitaciones

Muy útil para proyectos finales o documentación de entregables.



Generación de documentación técnica y comentarios de código.



3.2 — Tipos de documentación que puede generar la IA

◆ Guías de uso

La IA puede crear guías orientadas a usuarios:

- ☐ pasos para ejecutar
- ☐ ejemplos prácticos
- ☐ capturas descritas
- ☐ advertencias
- ☐ resolución de problemas

Facilita la comprensión del software por parte de terceros.



Generación de documentación técnica y comentarios de código.



3.2 — Tipos de documentación que puede generar la IA

◆ Guías de uso

La IA puede crear guías orientadas a usuarios:

- ☐ pasos para ejecutar
- ☐ ejemplos prácticos
- ☐ capturas descritas
- ☐ advertencias
- ☐ resolución de problemas

Facilita la comprensión del software por parte de terceros.



Generación de documentación técnica y comentarios de código.



3.2 — Tipos de documentación que puede generar la IA

◆ Documentación de APIs

La IA puede generar:

- ☐ descripción de endpoints
- ☐ parámetros
- ☐ respuestas
- ☐ códigos de estado
- ☐ ejemplos de peticiones y respuestas
- ☐ notas de seguridad

Esto ayuda a los profesionales a entender cómo se documenta una API profesional.



Generación de documentación técnica y comentarios de código.



3.2 — Tipos de documentación que puede generar la IA

✓ Documentación automática

Documentación generada a partir del código siguiendo estándares y herramientas profesionales.

◆ OpenAPI / Swagger

La IA puede:

- ☐ generar especificaciones OpenAPI
- ☐ describir endpoints
- ☐ definir esquemas
- ☐ documentar parámetros y respuestas
- ☐ detectar inconsistencias

Ideal para enseñar documentación de APIs REST.



Generación de documentación técnica y comentarios de código.



■ 3.2 — Tipos de documentación que puede generar la IA

◆ JSDoc / TypeDoc

La IA puede producir:

- ☐ anotaciones JSDoc completas
- ☐ documentación TypeDoc para TypeScript
- ☐ enlaces entre módulos
- ☐ ejemplos de uso

Esto permite generar documentación navegable automáticamente.



Generación de documentación técnica y comentarios de código.



3.2 — Tipos de documentación que puede generar la IA

◆ Sphinx (Python)

La IA puede:

- ☐ generar docstrings compatibles
- ☐ crear páginas de documentación
- ☐ estructurar secciones
- ☐ generar índices y módulos

Perfecto para proyectos Python más avanzados.

Generación de documentación técnica y comentarios de código.



■ 3.3 — Cómo pedir documentación a la IA

Un prompt debe incluir:

- El código completo
- El lenguaje y estilo deseado (Google, NumPy, JSDoc...)
- El nivel de detalle
- El público objetivo (alumno, desarrollador, usuario final)
- El formato (docstring, README, explicación paso a paso...)

Ejemplo:

"Genera un docstring estilo Google para esta función y explica su propósito, parámetros, retornos y errores posibles."



3.3 — Cómo pedir documentación a la IA

Genera documentación técnica para esta función Python:

CÓDIGO:

```
def calcular_imc(peso, altura):  
    if altura <= 0:  
        return None  
    imc = peso / (altura ** 2)  
    return round(imc, 2)
```

REQUERIMIENTOS:

1. Estilo de docstring: Google
2. Nivel de detalle: Medio (para estudiantes de programación)
3. Público objetivo: Alumnos de primer año de informática
4. Formato: Docstring completo + explicación didáctica breve
5. Incluir: propósito, fórmula matemática, ejemplos prácticos y errores comunes

INSTRUCCIÓN ESPECÍFICA:

"Explica como si fuera para alguien que está aprendiendo su primera función en Python"

Resultado Esperado:

Función Documentada:

```
def calcular_imc(peso, altura):
    """
    Calcula el Índice de Masa Corporal (IMC) a partir del peso y altura.

    El IMC es una medida utilizada para determinar si una persona tiene
    un peso saludable en relación con su estatura.

    Parámetros:
        peso (float): Peso en kilogramos. Debe ser un número positivo.
        altura (float): Altura en metros. Debe ser mayor que 0.

    Retorna:
        float | None: El IMC calculado redondeado a 2 decimales.
                      Retorna None si la altura es 0 o negativa.

    Ejemplos:
        >>> calcular_imc(70, 1.75)
        22.86

        >>> calcular_imc(85, 1.80)
        26.23

        >>> calcular_imc(60, 0)
        None

    Fórmula:
        IMC = peso / (altura²)
        donde:
            peso está en kilogramos
            altura está en metros

    Categorías de IMC (OMS):
        < 18.5: Bajo peso
        18.5 - 24.9: Peso normal
        25.0 - 29.9: Sobrepeso
        >= 30.0: Obesidad
    """
    if altura <= 0:
        return None
    imc = peso / (altura ** 2)
    return round(imc, 2)
```

3.3 — Cómo pedir documentación a la IA

¿Qué hace esta función?

1. Recibe tu peso (kg) y altura (m)
2. Aplica la fórmula matemática: $\text{peso} \div (\text{altura} \times \text{altura})$
3. Te devuelve tu IMC con 2 decimales

Si pesas 70kg y mides 1.75m:

1. $\text{altura}^2 = 1.75 \times 1.75 = 3.0625$

2. $70 \div 3.0625 = 22.857...$

3. Redondeamos: 22.86 ← ¡Este es tu IMC!

Errores que evita:

-  No permite altura = 0 (evita división por cero)
-  Redondea el resultado para mejor lectura
-  Devuelve None si hay datos inválidos

Elemento en el Prompt	¿Por qué es importante?	Ejemplo de texto
Rol (Role)	Define la perspectiva de la IA.	"Actúa como un experto documentador técnico ."
Contexto (Context)	Da información sobre el proyecto o situación.	"Estamos trabajando en un proyecto de e-commerce con microservicios."
Acción (Action)	Lo que quieres que la IA haga.	" Genera un docstring estilo Google para la siguiente función."
Formato (Format)	Cómo quieres que sea la salida.	" En formato YAML ." / " Con una tabla para los parámetros."
Restricciones (Constraints)	Limitaciones o requisitos específicos.	" No uses jerga técnica avanzada ." / " Incluye un ejemplo de uso ."
Audiencia (Audience)	Para quién es la documentación.	"La audiencia son estudiantes de primer año de programación ."

Generación de documentación técnica y comentarios de código.



3.4 — Ejemplo real: Docstring generado por IA

Prompt:

"Genera un docstring estilo Google."

Resultado esperado:

- Descripción clara
- Parámetros
- Tipo de retorno
- Errores posibles
- Ejemplo de uso

```
def calcular_total(precio, cantidad):  
    return precio * cantidad
```

Este prompt es breve, pero suficiente para que la IA genere documentación estructurada siguiendo el estándar Google Docstring.



Generación de documentación técnica y comentarios de código.



3.4 — Ejemplo real: Docstring generado por IA

✓ 1. Descripción clara

Debe explicar qué hace la función, en una o dos frases, sin repetir literalmente el nombre.

Ejemplo esperado:

Calcula el importe total multiplicando el precio unitario por la cantidad solicitada.

La IA debe evitar descripciones vagas como "Calcula algo".



```
def calcular_total(precio, cantidad):  
    return precio * cantidad
```

Generación de documentación técnica y comentarios de código.



3.4 — Ejemplo real: Docstring generado por IA

✓ 2. Parámetros

Debe documentar cada parámetro con:

- nombre
- tipo esperado
- descripción clara
- restricciones (si las hay)

Ejemplo esperado:

Args:

```
precio (float): Precio unitario del producto.  
cantidad (int): Número de unidades a calcular.
```



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

```
def calcular_total(precio, cantidad):  
    return precio * cantidad
```

Generación de documentación técnica y comentarios de código.



3.4 — Ejemplo real: Docstring generado por IA

✓ 3. Tipo de retorno

Debe indicar:

- tipo devuelto
- significado del valor
- posibles variaciones (por ejemplo, si puede devolver None)

Ejemplo esperado:

Returns:

`float: Importe total resultante de la operación.`



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

```
def calcular_total(precio, cantidad):  
    return precio * cantidad
```

Generación de documentación técnica y comentarios de código.



3.4 — Ejemplo real: Docstring generado por IA

✓ 4. Errores posibles

Incluso en funciones simples, la IA debe identificar riesgos:

- tipos incorrectos
- valores negativos
- valores nulos
- overflow o límites

Ejemplo esperado:

Raises:

`TypeError: Si precio o cantidad no son numéricos.`

`ValueError: Si cantidad es negativa.`



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

```
def calcular_total(precio, cantidad):  
    return precio * cantidad
```

Generación de documentación técnica y comentarios de código.



3.4 — Ejemplo real: Docstring generado por IA

✓ 5. Ejemplo de uso

La IA debe generar un ejemplo claro y ejecutable:

Example:

```
>>> calcular_total(5.0, 3)
15.0
```



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

```
def calcular_total(precio, cantidad):
    return precio * cantidad
```

Generación de documentación técnica y comentarios de código.



3.4 — Ejemplo real: Docstring generado por IA

Docstring completo esperado (estilo Google):

```
def calcular_total(precio, cantidad):  
    """  
    Calcula el importe total multiplicando el precio unitario por la cantidad.  
  
    Args:  
        precio (float): Precio unitario del producto.  
        cantidad (int): Número de unidades a calcular.  
  
    Returns:  
        float: Importe total resultante de la operación.  
  
    Raises:  
        TypeError: Si alguno de los parámetros no es numérico.  
        ValueError: Si la cantidad es negativa.  
  
    Example:  
        >>> calcular_total(5.0, 3)  
        15.0  
    """  
    return precio * cantidad
```



Generación de documentación técnica y comentarios de código.



3.4 — Ejemplo real: Docstring generado por IA

"Error común"

⚠ Error común: La IA puede "alucinar" parámetros

****Prompt:**** "Genera un docstring para `def get\_user(id)`."

****Salida incorrecta (posible):****

```python

```
def get_user(id):
```

```
 """
```

```
 Obtiene un usuario por ID.
```

```
 Args:
```

```
 id (int): ID del usuario.
```

```
 include_details (bool, optional): Si incluir detalles. Defaults to False. # ¡Este parámet
```

```
ro no existe!
```

```
 Returns:
```

```
 dict: Usuario.
```

```
 """
```

```
 # ...
```

La IA a veces inventa parámetros opcionales que no están en el código.

**¡Siempre debes verificar y corregir la salida!**

Generación de documentación técnica y comentarios de código.



## ■ 3.5 — Documentación de APIs con IA

La IA puede generar documentación de APIs de forma rápida, coherente y alineada con estándares profesionales.

Esto permite a los desarrolladores aprender cómo se documenta una API real sin necesidad de memorizar formatos complejos.

- ✓ 1. Esquemas OpenAPI
- ✓ 2. Descripciones de endpoints
- ✓ 3. Ejemplos de request/response
- ✓ 4. Códigos de error
- ✓ 5. Reglas de validación





## 3.5 — Documentación de APIs con IA



### 1. Esquemas OpenAPI



La IA puede producir especificaciones completas en formato OpenAPI 3.x, incluyendo:

- estructura del documento (`openapi`, `info`, `servers`)
- definición de endpoints
- parámetros de ruta, query y header
- cuerpos de petición (`requestBody`)
- esquemas de datos (`schemas`)
- respuestas con códigos de estado
- ejemplos integrados
- reglas de seguridad (`securitySchemes`)

Esto permite generar documentación compatible con Swagger UI o Redoc.

### Ejemplo de lo que puede generar:

- `paths: /login: post:`
- `components: schemas: User:`
- `security: bearerAuth`

## 3.5 — Documentación de APIs con IA

### ✓ 2. Descripciones de endpoints

La IA puede redactar descripciones claras y profesionales para cada endpoint:

- qué hace
- para qué sirve
- quién puede usarlo
- qué datos requiere
- qué devuelve
- qué errores puede producir

Esto ayuda a los alumnos a entender la intención de cada ruta.

### Ejemplo:

"Este endpoint permite autenticar a un usuario mediante email y contraseña, devolviendo un token JWT válido por 24 horas."



## 3.5 — Documentación de APIs con IA



### 3. Ejemplos de request/response



La IA puede generar ejemplos realistas y coherentes:

Ejemplo de request:

```
{
 "email": "usuario@example.com",
 "password": "123456"
}
```

Ejemplo de response:

```
{
 "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
 "expiresIn": 86400
}
```

Esto es clave para que los alumnos entiendan cómo interactuar con la API.

Generación de documentación técnica y comentarios de código.



## 3.5 — Documentación de APIs con IA

### ✓ 4. Códigos de error

La IA puede documentar:

- errores de validación (400)
- credenciales incorrectas (401)
- permisos insuficientes (403)
- recurso no encontrado (404)
- errores del servidor (500)

Y explicar cuándo se produce cada uno.

**Ejemplo:**

401 Unauthorized — Se devuelve cuando las credenciales no coinciden con ningún usuario registrado.



Generación de documentación técnica y comentarios de código.



## 3.5 — Documentación de APIs con IA

### ✓ 5. Reglas de validación

La IA puede describir las validaciones necesarias para cada campo:

- tipos
- longitudes mínimas y máximas
- formatos (email, UUID, fecha...)
- campos obligatorios
- restricciones de negocio

### Ejemplo:

"El campo `password` debe tener al menos 8 caracteres y contener una mayúscula."



Generación de documentación técnica y comentarios de código.



## 3.5 — Documentación de APIs con IA

### 🧩 Ejemplo de prompt

“Genera la documentación OpenAPI para este endpoint Express. Incluye descripción, parámetros, requestBody, respuestas, códigos de error y ejemplos.”  
Este prompt funciona porque:

- define el estándar (OpenAPI)
- indica el framework (Express)
- especifica qué secciones incluir
- pide ejemplos
- orienta a un resultado profesional

### Mensaje clave

La IA no solo documenta APIs: enseña a documentarlas siguiendo estándares profesionales como OpenAPI.  
Aun así, la documentación generada debe revisarse y adaptarse al proyecto real.



## 3.5 — Documentación de APIs con IA

### Estrategias de prompt avanzado

- \* **\*\*Encadenamiento de prompts:\*\*** Primero pide un resumen de la lógica, luego pide la documentación basada en ese resumen.
  - \* **\*Prompt 1:\*** "Explica la lógica de este endpoint de login."
  - \* **\*Prompt 2:\*** "Ahora, basándote en tu explicación, genera un docstring para este mismo endpoint en formato Google."
- \* **\*\*Usar ejemplos (few-shot learning):\*\*** Dale un ejemplo de cómo quieres la salida.
  - \* **\*Prompt:\*** "Genera la documentación para `get\_user`. Sigue EXACTAMENTE el formato de este ejemplo para `create\_user`: ..."
- \* **\*\*Pedir formatos estructurados:\*\*** Pide la salida en JSON, YAML o Markdown para integrarla fácilmente en otras herramientas.
  - \* **\*Prompt:\*** "Genera la documentación para esta API en formato JSON, siguiendo la estructura de OpenAPI."

```

app.post('/api/usuarios/login', async (req, res) => {
 try {
 const { email, password } = req.body;

 // Validaciones
 if (!email || !password) {
 return res.status(400).json({ error: 'Email y password requeridos' });
 }

 const usuario = await Usuario.findOne({ email });
 if (!usuario || !await usuario.compararPassword(password)) {
 return res.status(401).json({ error: 'Credenciales inválidas' });
 }

 const token = jwt.sign({ id: usuario._id }, process.env.JWT_SECRET,
 { expiresIn: '24h' });

 res.status(200).json({
 token,
 usuario: {
 id: usuario._id,
 nombre: usuario.nombre,
 email: usuario.email
 }
 });
 } catch (error) {
 res.status(500).json({ error: 'Error en el servidor' });
 }
});

```

“Genera la documentación OpenAPI para este endpoint Express.



## REQUERIMIENTOS:

1. Estándar: OpenAPI 3.0.0
2. Framework: Express.js
3. Secciones obligatorias:
  - Descripción del endpoint
  - Parámetros (si aplica)
  - Request Body con esquema JSON
  - Respuestas exitosas (200, 201)
  - Respuestas de error (400, 401, 500)
  - Ejemplos completos de request/response
4. Incluir: autenticación JWT en respuestas
5. Formato: YAML estructurado
6. Contexto: Sistema de autenticación de usuarios

**OBJETIVO:** Documentación profesional lista para usar en Swagger UI.”

# Código YAML

```
paths:
 /api/usuarios/login:
 post:
 tags:
 - Autenticación
 summary: Iniciar sesión de usuario
 description: |
 Endpoint para autenticar usuarios en el sistema.
 Valida credenciales y retorna un token JWT para autorización
 en endpoints protegidos.
 operationId: loginUsuario
 requestBody:
 required: true
 description: Credenciales del usuario
 content:
 application/json:
 schema:
 type: object
 required:
 - email
 - password
 properties:
 email:
 type: string
 format: email
 example: usuario@ejemplo.com
 description: Email registrado del usuario
 password:
 type: string
```

```
 format: password
 example: "MiContraseñaSegura123"
 description: Contraseña del usuario
 example:
 email: "ana.garcia@correo.com"
 password: "Password123!"
 responses:
 '200':
 description: Login exitoso
 content:
 application/json:
 schema:
 type: object
 properties:
 token:
 type: string
 description: Token JWT para autenticación
 example: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
 usuario:
 type: object
 properties:
 id:
 type: string
 example: "507f1f77bcf86cd799439011"
 nombre:
 type: string
 example: "Ana García"
 email:
 type: string
 example: "ana.garcia@correo.com"
```

US

# Código YAML



## 🔧 PROMPT ESTRUCTURADO:

"Genera documentación OpenAPI para este endpoint Express.

### INCLUYE:

- ✓ Descripción clara
- ✓ Parámetros del request
- ✓ Request Body con esquema
- ✓ Respuestas (200, 400, 401, 500)
- ✓ Ejemplos de uso
- ✓ Contexto de autenticación

ESTÁNDAR: OpenAPI 3.0.0

OBJETIVO: Swagger UI ready

```
'401':
 description: Credenciales incorrectas
 content:
 application/json:
 schema:
 type: object
 properties:
 error:
 type: string
 example: "Credenciales inválidas"
'500':
 description: Error interno del servidor
 content:
 application/json:
 schema:
 type: object
 properties:
 error:
 type: string
 example: "Error en el servidor"
security: []
```

✅ DOCUMENTACIÓN GENERADA:

- ✓ OpenAPI 3.0.0 compliant
- ✓ Esquemas JSON validados
- ✓ 4 respuestas HTTP documentadas
- ✓ Ejemplos reales de request/response
- ✓ Tags de organización
- ✓ Ready para Swagger UI

# Por qué este Prompt Funciona

| Elemento del Prompt          | ¿Qué logra?                | Resultado en la documentación       |
|------------------------------|----------------------------|-------------------------------------|
| "Genera OpenAPI"             | Define el estándar         | Documentación compatible            |
| "para Express"               | Contextualiza el framework | Middleware y estructura apropiada   |
| "incluye requestBody"        | Especifica contenido       | Schema JSON completo                |
| "respuestas y códigos error" | Cubre casos reales         | 400, 401, 500 documentados          |
| "pide ejemplos"              | Añade valor práctico       | Ejemplos de request/response        |
| "profesional"                | Eleva el nivel             | Documentación lista para producción |

Generación de documentación técnica y comentarios de código.



## ■ 3.6 — Generación de README con IA

Un buen README incluye:

- Descripción del proyecto
- Instalación
- Uso
- Ejemplos
- Estructura del repositorio
- Dependencias
- Licencia

**Prompt recomendado:**

“Genera un README profesional para este proyecto, incluyendo instalación, uso y ejemplos.”



Generación de documentación técnica y comentarios de código.



## 3.7 — IA explicando código heredado

✓ La IA puede analizar...

### ◆ Código antiguo

La IA puede revisar código escrito hace años:

- estilos obsoletos
- patrones que ya no se usan
- librerías desactualizadas
- funciones demasiado largas
- lógica duplicada

Y ofrecer una explicación moderna y comprensible.



Generación de documentación técnica y comentarios de código.



## ■ 3.7 — IA explicando código heredado

✓ La IA puede analizar...

◆ Código sin comentarios

Mucho código heredado carece de:

- comentarios
- docstrings
- documentación externa
- nombres descriptivos

La IA puede inferir la intención del autor y explicar qué hace cada parte.

Generación de documentación técnica y comentarios de código.



## ■ 3.7 — IA explicando código heredado

✓ La IA puede analizar...

◆ Código escrito por terceros

Cuando el alumno recibe código que no ha escrito:

- proyectos de compañeros
- repositorios externos
- código de prácticas anteriores
- módulos integrados de terceros

La IA puede actuar como un "traductor" que aclara la lógica y las decisiones de diseño

Generación de documentación técnica y comentarios de código.



## ■ 3.7 — IA explicando código heredado

✓ La IA puede analizar...

◆ Código complejo o mal estructurado

La IA puede analizar:

- funciones demasiado largas
- anidamientos profundos
- estructuras difíciles de seguir
- dependencias circulares
- patrones inconsistentes

Y ofrecer una explicación clara y ordenada.



Generación de documentación técnica y comentarios de código.



## ■ 3.7 — IA explicando código heredado

✓ Y generar...

### ◆ Explicaciones detalladas

La IA puede producir:

- descripciones paso a paso
- resúmenes de lógica
- interpretación de algoritmos
- aclaración de flujos internos
- explicación del propósito de cada módulo

Esto ayuda a los programadores a entender el “por qué” detrás del código.



Generación de documentación técnica y comentarios de código.



## ■ 3.7 — IA explicando código heredado

✓ Y generar...

### ◆ Diagramas conceptuales (en texto)

Aunque no genera imágenes, la IA puede crear diagramas conceptuales en texto:

flujos de datos

relaciones entre módulos

secuencias de ejecución

dependencias entre funciones

diagramas de responsabilidades

Ejemplo:

```
[Controlador] → [Servicio] → [Repositorio] → [Base de datos]
```

Esto facilita visualizar la arquitectura sin herramientas externas.



Generación de documentación técnica y comentarios de código.



## ■ 3.7 — IA explicando código heredado

✓ Y generar...

### ◆ Resúmenes de módulos

La IA puede analizar un archivo completo y generar:

- propósito del módulo
- funciones principales
- dependencias internas
- puntos críticos
- riesgos o malas prácticas

Ideal para proyectos grandes o mal documentados.

Generación de documentación técnica y comentarios de código.



## 3.7 — IA explicando código heredado

✓ Y generar...

### ◆ Dependencias

La IA puede identificar:

- qué módulos importan a cuáles
- qué funciones dependen de otras
- qué librerías externas se usan
- qué partes del código están acopladas
- qué dependencias son innecesarias

### Mensaje clave

La IA convierte el código heredado en algo comprensible, explicando su lógica, estructura y dependencias.

Aun así, debemos validar siempre la explicación y contrastarla con el comportamiento real del código.



## 3.7 — IA explicando código heredado

```
def f(a, b):
 c = []
 for i in range(len(a)):
 if i < len(b):
 c.append(a[i] * b[i])
 else:
 c.append(a[i])
 return sum(c) / len(c)
```

**\*\*Explicación generada por IA:\*\***

Esta función calcula la **\*\*media ponderada de dos listas\*\***. Su propósito es combinar los valores de ``a`` y ``b``, dando prioridad a ``b`` cuando exista.

1. **\*\*Entrada:\*\*** Recibe dos listas, ``a`` y ``b`` (probablemente de números).
2. **\*\*Combinación:\*\*** Itera sobre los elementos de ``a`` usando su índice ``i``.
  - \* Si ``b`` tiene un elemento en la misma posición ``i``, multiplica el valor de ``a[i]`` por ``b[i]``.
  - \* Si ``b`` es más corta y no tiene un elemento en ``i``, solo toma el valor de ``a[i]``.
3. **\*\*Resultado:\*\*** Acumula todos estos valores en una nueva lista ``c``.
4. **\*\*Salida:\*\*** Finalmente, devuelve la **\*\*media (promedio)\*\*** de los valores almacenados en ``c``.

Generación de documentación técnica y comentarios de código.



## ■ 3.8 — Buenas prácticas al documentar con IA y errores comunes

- Revisar siempre la documentación generada
- Ajustar el nivel de detalle
- Mantener consistencia en el estilo
- Evitar comentarios obvios
- Documentar decisiones, no solo líneas
- Actualizar documentación tras cambios
- Indicar qué partes fueron generadas por IA
- Pedir documentación sin incluir el código
- No especificar el estilo
- Aceptar docstrings incorrectos
- Documentar lo obvio
- No revisar ejemplos generados
- No adaptar la documentación al público objetivo





Gracias.



MINISTERIO  
DE EDUCACIÓN, FORMACIÓN PROFESIONAL  
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN  
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



